# Run WSO2 Identity Server 7.3 on AWS EKS with an existing PostgreSQL (dev environment)

This guide sets up WSO2 Identity Server 7.3.0 on Amazon EKS for a development environment, using a **PostgreSQL server you already have**. The server can be Amazon RDS, Aurora PostgreSQL, PostgreSQL on EC2, or PostgreSQL running in a cluster. When you finish, you have:

- WSO2 behind an AWS Network Load Balancer (NLB), at the NLB's DNS name, using your wildcard certificate for `*.elb.<region>.amazonaws.com`;
- WSO2's data in new databases on your existing PostgreSQL server, so nothing is lost when the pod restarts;
- your own token-signing key, instead of the default key that ships in the WSO2 image;
- passwords and keys kept in AWS Secrets Manager, not in the image or in git.

If you'd rather create a new RDS instance just for WSO2, use the companion guide *Run WSO2 Identity Server 7.3 on AWS EKS (dev environment)*. Only the database sections differ.

## Before you start

You need:

- an EKS cluster, with `kubectl` access to it;
- the AWS Load Balancer Controller installed in the cluster;
- the External Secrets Operator installed, with a `ClusterSecretStore` for AWS Secrets Manager (optional: §4 shows a manual alternative);
- an ECR repository for the WSO2 image;
- your wildcard certificate for `*.elb.<region>.amazonaws.com`, as files or already in ACM (§2);
- a database administrator who can create databases and a user on the existing PostgreSQL server, or that access yourself;
- `openssl`, `docker`, `psql` and the AWS CLI on an admin workstation.

Replace these values everywhere in this guide:

| Placeholder | Meaning |
|---|---|
| `<nlb-dns-name>` | The DNS name AWS gives WSO2's load balancer, for example `wso2-dev-0123456789abcdef.elb.ap-south-1.amazonaws.com`. You get it in §8.1 |
| `<region>`, `<account>` | Your AWS region and account ID |
| `<db-host>`, `<db-port>` | Your PostgreSQL server's host name and port (usually `5432`). Use the name that matches the server's TLS certificate |

| Placeholder | Meaning |
|---|---|
| `db-ca.pem` | The certificate authority that signed your PostgreSQL server's TLS certificate (§3.2) |
| `wso2` | The Kubernetes namespace |

# 1. Choose your setup

| Topic | Recommended for dev | Reason |
|---|---|---|
| Load balancer | An **NLB**: it either ends TLS with your certificate from ACM, or passes TLS through to WSO2 (§8) | Your wildcard certificate covers NLB names (`*.elb.<region>.amazonaws.com`), not ALB names |
| WSO2's address | The NLB's DNS name, `https://<nlb-dns-name>`, used by everyone, even from inside the cluster | Your certificate covers that name, and tokens name this address as their issuer |
| Keep the address fixed | Create the NLB first and never delete it (§8.1) | A new NLB gets a new name, which changes WSO2's address and token issuer |
| Trust in your certificate | Browsers and clients that call WSO2 must trust your certificate (`wso2-trust.pem`, §2.1). WSO2 must trust it too (§5) | It isn't from a public authority, so default trust stores don't know it |
| Internal or public | An internal load balancer if all users are on your network or VPN. Otherwise, internet-facing and limited to your office IP ranges | Browsers must reach WSO2's sign-in pages |
| Database | **Three new databases** on your existing PostgreSQL server, owned by a new `wso2` user | They keep WSO2 apart from your other applications' data. Agents must have their own database (§3.1) |
| Number of pods | **1** | Running more than one pod needs WSO2 clustering (§10) |
| Token-signing key | A **new** key, used only for signing | The WSO2 image ships with a default key that is the same everywhere. Never reuse your TLS key for signing |

```
Users and applications
       |  https://<nlb-dns-name>   (wso2-dev-….elb.<region>.amazonaws.com)
       ▼
NLB: A) ends TLS with your ACM certificate, or B) passes TLS through
       |  TLS to port 9443
       ▼
EKS: Service "wso2is-public" ──► WSO2 pod (1 replica)
       |   TLS keystore: your wildcard certificate (B) or a backend certificate (A)
       |   Primary keystore: your token-signing key
       ▼
Your existing PostgreSQL server: new databases wso2_shared, wso2_identity, wso2_agent
```

---

# 2. Prepare the certificates

## 2.1 Your wildcard certificate

Your certificate is for `*.elb.<region>.amazonaws.com`. That pattern matches the DNS names AWS gives **Network Load Balancers (NLBs)** in your region, for example `wso2-dev-0123456789abcdef.elb.ap-south-1.amazonaws.com`. It doesn't match Application Load Balancer names, which look like `name-1234567890.ap-south-1.elb.amazonaws.com`. So:

- **put WSO2 behind an NLB;**
- **use the NLB's DNS name as WSO2's address.**

A wildcard covers one level only: `anything.elb.<region>.amazonaws.com` matches, but `a.b.elb.<region>.amazonaws.com` doesn't. NLB names always have exactly one level there.

Collect these files:

| File | Contents | Needed for |
|---|---|---|
| `tls.crt` | The wildcard certificate | Both options in §8 |
| `tls.key` | Its private key, without a password | Option B only |
| `chain.pem` | Intermediate certificates, only if a CA that uses them issued your certificate | If you have one |
| `wso2-trust.pem` | What clients must trust. For a **self-signed** certificate, it's `tls.crt` itself. For a certificate from your organization's CA, it's that CA's root certificate. It isn't secret | Both options |

**Self-signed or issued by a CA?** Run:

```
openssl x509 -in tls.crt -noout -subject -issuer -enddate -ext subjectAltName
```

If the subject and the issuer are the same, the certificate is self-signed. In that case:

- there is no `chain.pem`;
- copy the certificate as the trust file: `cp tls.crt wso2-trust.pem`.

Clients then trust the certificate itself.

**Check the files.** The first command is for a self-signed certificate, the second for one issued by your CA:

```
openssl verify -CAfile wso2-trust.pem tls.crt
openssl verify -CAfile wso2-trust.pem -untrusted chain.pem tls.crt
```

If you have the key, confirm it belongs to the certificate. The two commands must print the same value:

```
openssl x509 -in tls.crt -noout -pubkey | openssl sha256
openssl pkey -in tls.key -pubout | openssl sha256
```

**The certificate in ACM (Option A).** It may already be in ACM. Look for it, and note its ARN:

```
aws acm list-certificates --region <region> --query "CertificateSummaryList[].
[DomainName,CertificateArn]" --output table
```

If its key type isn't RSA 2048, add `--includes keyTypes=RSA_4096,EC_prime256v1` to see it. If it isn't in ACM yet and you have the key, import it (leave out `--certificate-chain` if you have no `chain.pem`):

```
aws acm import-certificate --certificate fileb://tls.crt --private-key fileb://tls.key --
certificate-chain fileb://chain.pem --region <region>
```

**WSO2's own TLS keystore.** WSO2 always serves a certificate on its port 9443. The name `wso2-tls` must match the TLS keystore alias in §6.

- **Option B (the NLB passes TLS straight through):** WSO2 must serve your wildcard certificate. Build the keystore from it. Leave out `-certfile chain.pem` if you have no chain.

  ```
  openssl pkcs12 -export -name wso2-tls -inkey tls.key -in tls.crt -certfile chain.pem -
  passout pass:'<keystore password>' -out tls.p12
  ```

- **Option A (the NLB ends TLS with the ACM certificate):** clients see the ACM certificate. The NLB doesn't check WSO2's certificate on the hop inside AWS, so if you don't have the wildcard key, create a separate certificate just for that hop:

  ```
  openssl req -x509 -newkey rsa:2048 -nodes -sha256 -days 825 -keyout backend.key -out
  backend.crt -subj "/CN=wso2-backend"
  openssl pkcs12 -export -name wso2-tls -inkey backend.key -in backend.crt -passout
  pass:'<keystore password>' -out tls.p12
  ```

  Then always reach WSO2 through the NLB. Never send clients to the pod or the Kubernetes Service directly.

## 2.2 The token-signing key

WSO2 signs every token it issues with this key. Applications check tokens with its public half, which WSO2 publishes at `/oauth2/jwks`.

```
openssl req -x509 -new -newkey rsa:2048 -nodes -sha256 -days 1825 -keyout signing.key -out signing.crt -subj "/CN=WSO2 token signing (dev)"
openssl pkcs12 -export -name wso2carbon -inkey signing.key -in signing.crt -passout pass:'<keystore password>' -out signing.p12
```

- **The name must match:** `wso2carbon` must match the primary keystore alias in §6.
- **Self-signed is fine:** applications trust this key through `/oauth2/jwks`, not through a certificate chain.
- **Store it, then delete the local copies:** put the files in Secrets Manager (§4) and remove them from the workstation.

---

# 3. Prepare your existing PostgreSQL server

## 3.1 Check the server first

| Check | What to look for |
|---|---|
| Version | WSO2 7.3.0 was tested with PostgreSQL 16. For other versions, check WSO2's list of supported databases for 7.3 |
| Network | The EKS pods must reach `<db-host>:<db-port>`. Allow the EKS nodes' (or pods') security group in the database's security group or firewall. If the server is outside the VPC (another VPC, or on-premises), check the peering, Transit Gateway or VPN route, and keep the delay low: WSO2 queries the database on every sign-in |
| Connections | WSO2 opens a pool of connections to each of its three databases. Check that `max_connections` has room: run `SHOW max_connections;` and `SELECT count(*) FROM pg_stat_activity;`. If room is tight, limit WSO2's pools (§6) |
| TLS | The server should accept TLS connections. Find out which certificate authority signed its certificate (§3.2) |
| Backups | Make sure the server's backup covers the new databases. A per-database backup job won't include them automatically |
| Extensions | None needed: WSO2's scripts create only tables, indexes and sequences |

**Why three databases, and never one?** WSO2 7.3 keeps agent identities (WSO2 Agents) in their own store. Its tables have the **same names** as tables in the shared database (`UM_USER`, `UM_ROLE`, `UM_USER_ATTRIBUTE`, `UM_USER_ROLE`), so the agent tables must live in a database of their own. Keep all three separate so that each one can be backed up and restored on its own.

If your database administrator can give you only two databases, the shared and identity tables don't overlap, so they could share one database, with agents in the second. This layout was not tested.

## 3.2 Find the database's certificate authority

WSO2 checks the database server's certificate. Save the authority's certificate as `db-ca.pem`:

| Your server | `db-ca.pem` is |
|---|---|
| Amazon RDS or Aurora | The RDS bundle, from https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem |
| PostgreSQL with a certificate from your organization | Your organization's root CA (ask your database or security team) |
| PostgreSQL in a cluster, with a certificate from cert-manager | The issuing authority's `ca.crt` |

To check the file works, run this from a host that can reach the database. It should connect without errors:

```
psql "host=<db-host> port=<db-port> dbname=postgres user=<admin user> sslmode=verify-full sslrootcert=db-ca.pem" -c "select version();"
```

## 3.3 Create the databases and the user

Run this as an administrator on your existing server (or give it to your database administrator):

```
CREATE ROLE wso2 LOGIN PASSWORD '<db password>';
CREATE DATABASE wso2_shared   OWNER wso2;   -- users, roles, shared settings
CREATE DATABASE wso2_identity OWNER wso2;   -- applications, tokens, sessions, connections
CREATE DATABASE wso2_agent    OWNER wso2;   -- agent identities (WSO2 Agents)

-- Only the wso2 user may connect to these databases.
-- (By default, PostgreSQL lets every user on the server connect to a new database.)
REVOKE ALL ON DATABASE wso2_shared, wso2_identity, wso2_agent FROM PUBLIC;
```

**Sharing the server with other projects is fine.** WSO2 stays separate from them:

- **Its own databases:** WSO2 uses only `wso2_shared`, `wso2_identity` and `wso2_agent`. It doesn't touch other projects' databases.
- **Its own user:** `wso2` owns these three databases and has no rights anywhere else. It can't read other projects' tables.
- **Kept closed:** the `REVOKE` line stops other projects' users from connecting to WSO2's databases.

Points to agree with your database administrator:

- **Names:** if these database names are already taken on the server, choose others (for example `iam_dev_shared`) and use them everywhere in this guide.
- **Connections:** WSO2 adds its own connections to the server's total (§3.1).
- **Maintenance:** restarting or upgrading the server interrupts WSO2 as well as the other projects.
- **Optional:** if your organization requires it, allow the `wso2` user only from the EKS network, and only to these databases, in `pg_hba.conf` or the equivalent setting.

### 3.4 Load the tables (once, into the new, empty databases)

The table scripts are inside the WSO2 image. Copy them out:

```
mkdir dbscripts
docker run --rm --entrypoint sh -v "$PWD/dbscripts:/out" wso2/wso2is:7.3.0 -c 'cd
$WSO2_SERVER_HOME/dbscripts && cp postgresql.sql /out/shared.sql && cp
identity/postgresql.sql /out/identity.sql && cp consent/postgresql.sql /out/consent.sql &&
cp identity/agent/postgresql.sql /out/agent.sql'
```

Run them as the `wso2` user, so it owns the tables:

```
export PGHOST=<db-host> PGPORT=<db-port> PGUSER=wso2 PGPASSWORD='<db password>'
PGSSLMODE=verify-full PGSSLROOTCERT=db-ca.pem
psql -v ON_ERROR_STOP=1 -q -d wso2_shared   -f dbscripts/shared.sql
psql -v ON_ERROR_STOP=1 -q -d wso2_identity -f dbscripts/identity.sql
psql -v ON_ERROR_STOP=1 -q -d wso2_identity -f dbscripts/consent.sql
psql -v ON_ERROR_STOP=1 -q -d wso2_agent    -f dbscripts/agent.sql
```

> **Warning:** the scripts begin by dropping their tables. On a shared server this matters even more:
>
> - Run them only in the three new databases.
> - Check the `-d` value on every line.
> - Never run them against another application's database, or against a database that WSO2 is already using.

---

## 4. Store the secrets

Create one secret in AWS Secrets Manager, for example `dev/wso2/identity-server`, with these keys:

| Key | Value |
|---|---|
| `admin-username` | The WSO2 super admin's username, for example `wso2admin` |
| `admin-password` | 8–30 characters, with upper and lower case letters, a digit and one of `!@#%&*` |
| `db-password` | The `wso2` database user's password |
| `keystore-password` | The password you used for `tls.p12` and `signing.p12` |
| `tls.p12` | `tls.p12`, base64-encoded |
| `signing.p12` | `signing.p12`, base64-encoded |
| `signing.crt` | The contents of `signing.crt` |

Copy it into a Kubernetes Secret named `wso2is` with the External Secrets Operator:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
```

```
metadata: {name: wso2is, namespace: wso2}
spec:
  refreshInterval: 1h
  secretStoreRef: {kind: ClusterSecretStore, name: aws-secrets-manager}
  target:
    name: wso2is
    template:
      engineVersion: v2
      data:
        admin-username: "{{ .adminUsername }}"
        admin-password: "{{ .adminPassword }}"
        db-password: "{{ .dbPassword }}"
        keystore-password: "{{ .keystorePassword }}"
        signing.crt: "{{ .signingCrt }}"
        tls.p12: "{{ .tlsP12 | b64dec }}"
        signing.p12: "{{ .signingP12 | b64dec }}"
  data:
    - {secretKey: adminUsername,    remoteRef: {key: dev/wso2/identity-server, property:
admin-username}}
    - {secretKey: adminPassword,    remoteRef: {key: dev/wso2/identity-server, property:
admin-password}}
    - {secretKey: dbPassword,       remoteRef: {key: dev/wso2/identity-server, property: db-
password}}
    - {secretKey: keystorePassword, remoteRef: {key: dev/wso2/identity-server, property:
keystore-password}}
    - {secretKey: signingCrt,       remoteRef: {key: dev/wso2/identity-server, property:
signing.crt}}
    - {secretKey: tlsP12,           remoteRef: {key: dev/wso2/identity-server, property:
tls.p12}}
    - {secretKey: signingP12,       remoteRef: {key: dev/wso2/identity-server, property:
signing.p12}}
```

Without the operator, you can create the same Secret by hand for a first test. Don't save it in git.

```
kubectl -n wso2 create secret generic wso2is \
  --from-literal=admin-username=wso2admin --from-literal=admin-password='…' \
  --from-literal=db-password='…' --from-literal=keystore-password='…' \
  --from-file=tls.p12 --from-file=signing.p12 --from-file=signing.crt
```

---

# 5. Build the WSO2 image

The official image doesn't include a PostgreSQL driver. Build a small image on top of it.

`start.sh` copies your keystores into place. It makes WSO2 trust your new signing certificate instead of the default one, and adds `wso2-trust.pem` to the certificates WSO2 trusts:

```
#!/bin/sh
set -eu
SEC="${WSO2_SERVER_HOME}/repository/resources/security"
cp /keys/signing.p12 /keys/tls.p12 "$SEC/"
keytool -delete -alias wso2carbon -keystore "$SEC/client-truststore.p12" -storepass
wso2carbon >/dev/null 2>&1 || true
keytool -importcert -noprompt -alias wso2carbon -file /keys/signing.crt \
  -keystore "$SEC/client-truststore.p12" -storepass wso2carbon >/dev/null
# Trust your certificate (or its CA), for calls from WSO2 to other services that use it
keytool -delete -alias wso2-trust -keystore "$SEC/client-truststore.p12" -storepass
wso2carbon >/dev/null 2>&1 || true
keytool -importcert -noprompt -alias wso2-trust -file /trust/wso2-trust.pem \
  -keystore "$SEC/client-truststore.p12" -storepass wso2carbon >/dev/null
exec /home/wso2carbon/docker-entrypoint.sh "$@"
```

`Dockerfile`:

```
FROM wso2/wso2is:7.3.0
ADD --chown=wso2carbon:wso2
https://repo1.maven.org/maven2/org/postgresql/postgresql/42.7.4/postgresql-42.7.4.jar \
    /home/wso2carbon/wso2is-7.3.0/repository/components/lib/
COPY --chown=wso2carbon:wso2 start.sh /home/wso2carbon/start.sh
ENTRYPOINT ["sh", "/home/wso2carbon/start.sh"]
```

Build and push:

```
aws ecr get-login-password --region <region> | docker login --username AWS --password-stdin
<account>.dkr.ecr.<region>.amazonaws.com
docker build -t <account>.dkr.ecr.<region>.amazonaws.com/wso2is-custom:7.3.0-1 .
docker push <account>.dkr.ecr.<region>.amazonaws.com/wso2is-custom:7.3.0-1
```

With `start.sh`, WSO2 trusts your certificate as well as the common public CAs. If WSO2 must call a server whose certificate comes from somewhere else that isn't public, import that certificate or CA in `start.sh` the same way, under its own alias. The database connection doesn't need this: it uses `db-ca.pem` directly (§6).

---

# 6. Write the configuration

Save this as `deployment.toml`. It goes into a ConfigMap in §7.

**First create the load balancer (§8.1) to get `<nlb-dns-name>`.** WSO2 needs its final address before its first start.

```
[server]
hostname = "<nlb-dns-name>"
node_ip = "127.0.0.1"
base_path = "https://$ref{server.hostname}"

# WSO2 listens on port 9443, but users reach it on 443 through the load balancer.
# This makes WSO2 publish https://<nlb-dns-name>/... with no port.
```

```
[transport.https.properties]
proxyPort = 443

[super_admin]
username = "$env{WSO2_ADMIN_USERNAME}"
password = "$env{WSO2_ADMIN_PASSWORD}"
create_admin_account = true

[user_store]
type = "database_unique_id"

[database.identity_db]
type = "postgre"
url = "jdbc:postgresql://<db-host>:<db-port>/wso2_identity?sslmode=verify-
full&sslrootcert=/home/wso2carbon/db-ca/db-ca.pem"
username = "wso2"
password = "$env{WSO2_DB_PASSWORD}"
driver = "org.postgresql.Driver"

[database.shared_db]
type = "postgre"
url = "jdbc:postgresql://<db-host>:<db-port>/wso2_shared?sslmode=verify-
full&sslrootcert=/home/wso2carbon/db-ca/db-ca.pem"
username = "wso2"
password = "$env{WSO2_DB_PASSWORD}"
driver = "org.postgresql.Driver"

# WSO2 Agents have their own store and database.
[datasource.AgentIdentity]
id = "AgentIdentity"
url = "jdbc:postgresql://<db-host>:<db-port>/wso2_agent?sslmode=verify-
full&sslrootcert=/home/wso2carbon/db-ca/db-ca.pem"
username = "wso2"
password = "$env{WSO2_DB_PASSWORD}"
driver = "org.postgresql.Driver"

# Token-signing key
[keystore.primary]
file_name = "signing.p12"
type = "PKCS12"
password = "$env{WSO2_KEYSTORE_PASSWORD}"
alias = "wso2carbon"
key_password = "$env{WSO2_KEYSTORE_PASSWORD}"

# TLS certificate
[keystore.tls]
file_name = "tls.p12"
type = "PKCS12"
password = "$env{WSO2_KEYSTORE_PASSWORD}"
alias = "wso2-tls"
key_password = "$env{WSO2_KEYSTORE_PASSWORD}"

[truststore]
```

```
file_name = "client-truststore.p12"
password = "wso2carbon"
type = "PKCS12"

# The two settings below are from WSO2's default configuration. Keep them.
[account_recovery.endpoint.auth]
hash = "66cd9688a2ae068244ea01e70f0e230f5623b7fa4cdecb65070a09ec06452262"

[identity.auth_framework.endpoint]
app_password = "dashboard"
```

**If the server is short of connections**, add pool limits under each database section. For example:

```
[database.identity_db.pool_options]
maxActive = 40
minIdle = 5

[database.shared_db.pool_options]
maxActive = 20
minIdle = 2

[datasource.AgentIdentity.pool_options]
maxActive = 10
minIdle = 2
```

Size them to the room you found in §3.1. These pool settings were not tested.

---

# 7. Deploy to Kubernetes

Save the following as `wso2is.yaml`. Paste your `deployment.toml` and `db-ca.pem` where shown.

```
apiVersion: v1
kind: Namespace
metadata: {name: wso2}
---
apiVersion: v1
kind: ConfigMap
metadata: {name: wso2is-config, namespace: wso2}
data:
  deployment.toml: |
    # paste deployment.toml from §6 here
---
apiVersion: v1
kind: ConfigMap
metadata: {name: db-ca, namespace: wso2}
data:
  db-ca.pem: |
    # paste db-ca.pem from §3.2 here
---
apiVersion: v1
kind: ConfigMap
```

```yaml
metadata: {name: wso2-trust, namespace: wso2}
data:
  wso2-trust.pem: |
    # paste wso2-trust.pem from §2.1 here
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: wso2is, namespace: wso2}
spec:
  replicas: 1
  strategy: {type: Recreate}
  selector: {matchLabels: {app: wso2is}}
  template:
    metadata:
      labels: {app: wso2is}
    spec:
      securityContext:
        runAsUser: 802          # the image's wso2carbon user
        runAsGroup: 802
        fsGroup: 802            # lets that user read the mounted key files
      containers:
        - name: wso2is
          image: <account>.dkr.ecr.<region>.amazonaws.com/wso2is-custom:7.3.0-1
          ports: [{name: https, containerPort: 9443}]
          env:
            - {name: WSO2_ADMIN_USERNAME,    valueFrom: {secretKeyRef: {name: wso2is, key:
admin-username}}}
            - {name: WSO2_ADMIN_PASSWORD,    valueFrom: {secretKeyRef: {name: wso2is, key:
admin-password}}}
            - {name: WSO2_DB_PASSWORD,       valueFrom: {secretKeyRef: {name: wso2is, key:
db-password}}}
            - {name: WSO2_KEYSTORE_PASSWORD, valueFrom: {secretKeyRef: {name: wso2is, key:
keystore-password}}}
            - {name: JAVA_OPTS, value: "-Xms1024m -Xmx2048m"}
          resources:
            requests: {cpu: "1", memory: 2Gi}
            limits: {memory: 3Gi}
          volumeMounts:
            - {name: config, mountPath: /home/wso2carbon/wso2-config-
volume/repository/conf/deployment.toml, subPath: deployment.toml}
            - {name: keys, mountPath: /keys, readOnly: true}
            - {name: wso2-trust, mountPath: /trust, readOnly: true}
            - {name: db-ca, mountPath: /home/wso2carbon/db-ca, readOnly: true}
          startupProbe:
            httpGet: {scheme: HTTPS, path: /api/health-check/v1.0/health, port: 9443}
            periodSeconds: 10
            failureThreshold: 30    # WSO2 needs 1–3 minutes to start
          readinessProbe:
            httpGet: {scheme: HTTPS, path: /api/health-check/v1.0/health, port: 9443}
            periodSeconds: 10
          livenessProbe:
            httpGet: {scheme: HTTPS, path: /api/health-check/v1.0/health, port: 9443}
            periodSeconds: 20
```

```
            failureThreshold: 6
      volumes:
        - name: config
          configMap: {name: wso2is-config}
        - name: wso2-trust
          configMap: {name: wso2-trust}
        - name: keys
          secret:
            secretName: wso2is
            defaultMode: 0440
            items:
              - {key: tls.p12, path: tls.p12}
              - {key: signing.p12, path: signing.p12}
              - {key: signing.crt, path: signing.crt}
        - name: db-ca
          configMap: {name: db-ca}
---
apiVersion: v1
kind: Service
metadata: {name: wso2is, namespace: wso2}
spec:
  selector: {app: wso2is}
  ports: [{name: https, port: 9443, targetPort: 9443}]
```

Notes:

- **Where the config goes:** the image copies any file under `/home/wso2carbon/wso2-config-volume/` over its own configuration. That's how your `deployment.toml` takes effect.
- **File permissions:** WSO2 runs as user 802, not as root. Without `fsGroup: 802` and `defaultMode: 0440`, it can't read the key files and stops at start-up.
- **Probe certificates:** Kubernetes health probes don't check certificates. That's normal and doesn't affect your users.
- **A database inside the cluster:** if your PostgreSQL runs in the same cluster, `<db-host>` is its service name (for example `postgres.data.svc.cluster.local`). That name must appear in the database's TLS certificate.

---

# 8. Expose WSO2 through an NLB

## 8.1 Create the load balancer first

WSO2's address is the NLB's DNS name, and WSO2 must know that address before its first start (§6). So create the NLB before you deploy WSO2. It doesn't matter that no WSO2 pod exists yet.

Choose one option:

| Option | Use it when | How TLS works |
|---|---|---|
| **A: the NLB ends TLS** | Your certificate is in ACM (you may not have its key) | The NLB shows the ACM certificate to clients, then opens a new TLS connection to WSO2 on port 9443 |

| Option | Use it when | How TLS works |
| --- | --- | --- |
| **B: TLS passthrough** | You have the certificate's key | The NLB passes the connection through, and WSO2 shows your certificate itself |

Save the matching Service as `nlb-service.yaml`.

**Option A:**

```
apiVersion: v1
kind: Service
metadata:
  name: wso2is-public
  namespace: wso2
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
    service.beta.kubernetes.io/aws-load-balancer-scheme: internal          # or internet-
facing
    service.beta.kubernetes.io/aws-load-balancer-name: wso2-dev
    service.beta.kubernetes.io/aws-load-balancer-attributes:
deletion_protection.enabled=true
    service.beta.kubernetes.io/aws-load-balancer-ssl-cert: arn:aws:acm:<region>:
<account>:certificate/<id>
    service.beta.kubernetes.io/aws-load-balancer-ssl-ports: "443"
    service.beta.kubernetes.io/aws-load-balancer-ssl-negotiation-policy: ELBSecurityPolicy-
TLS13-1-2-2021-06
    service.beta.kubernetes.io/aws-load-balancer-backend-protocol: ssl
    service.beta.kubernetes.io/aws-load-balancer-healthcheck-protocol: HTTPS
    service.beta.kubernetes.io/aws-load-balancer-healthcheck-path: /api/health-
check/v1.0/health
spec:
  type: LoadBalancer
  selector: {app: wso2is}
  ports: [{name: https, port: 443, targetPort: 9443, protocol: TCP}]
  # loadBalancerSourceRanges: [203.0.113.0/24]   # your office ranges, if internet-facing
```

**Option B:**

```
apiVersion: v1
kind: Service
metadata:
  name: wso2is-public
  namespace: wso2
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
    service.beta.kubernetes.io/aws-load-balancer-scheme: internal          # or internet-
facing
    service.beta.kubernetes.io/aws-load-balancer-name: wso2-dev
    service.beta.kubernetes.io/aws-load-balancer-attributes:
deletion_protection.enabled=true
```

```
    service.beta.kubernetes.io/aws-load-balancer-healthcheck-protocol: HTTPS
    service.beta.kubernetes.io/aws-load-balancer-healthcheck-path: /api/health-
check/v1.0/health
spec:
  type: LoadBalancer
  selector: {app: wso2is}
  ports: [{name: https, port: 443, targetPort: 9443, protocol: TCP}]
  # loadBalancerSourceRanges: [203.0.113.0/24]   # your office ranges, if internet-facing
```

Create it and read its DNS name:

```
kubectl create namespace wso2
kubectl apply -f nlb-service.yaml
kubectl -n wso2 get svc wso2is-public -w          # wait until EXTERNAL-IP shows a name,
then Ctrl+C
kubectl -n wso2 get svc wso2is-public -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

The name looks like `wso2-dev-0123456789abcdef.elb.ap-south-1.amazonaws.com`. Check that it ends in `.elb.<region>.amazonaws.com`, so your wildcard covers it. This is `<nlb-dns-name>`: put it in `deployment.toml` (§6).

You don't need a DNS record: AWS's name already resolves. An internal NLB's name resolves to private addresses, which work only from inside your network.

**Keep this load balancer for the life of the environment.** If the `wso2is-public` Service or the `wso2` namespace is deleted, AWS creates a new NLB with a **new** name. WSO2's address and token issuer would then change, and anything configured with the old address would need updating.
`deletion_protection.enabled=true` stops the NLB from being deleted by mistake; leave the Service out of clean-up scripts too.

## 8.2 Deploy WSO2

```
kubectl apply -f external-secret.yaml       # or create the Secret by hand (§4)
kubectl apply -f wso2is.yaml                # with <nlb-dns-name> in deployment.toml
kubectl -n wso2 rollout status deploy/wso2is --timeout=10m
kubectl -n wso2 logs deploy/wso2is | grep -E "ERROR|Carbon started"
```

You should see `WSO2 Carbon started in …` and no `ERROR` lines. A minute or two later, the NLB marks the pod healthy and starts sending traffic.

## 8.3 Later: an address in your own domain

The NLB name works, but WSO2's address then depends on that one load balancer. If you later get a DNS name in your own domain (for example `iam.dev.yourcompany.com`) with a certificate for it, you can move WSO2 to it: change `hostname` in §6 and point the DNS name at the NLB. This also changes the token issuer, so do it early, before other systems are set up with the old address.

---

# 9. Check that it works

Run these from a workstation that can reach the NLB, pointing the tools at `wso2-trust.pem` as shown. If the certificate is already trusted on the workstation, you can leave out `-CAfile` and `--cacert`. Don't use `-k` or `--insecure`.

| Check | Command | Expected result |
|---|---|---|
| Your certificate is shown | `openssl s_client -connect <nlb-dns-name>:443 -servername <nlb-dns-name> -CAfile wso2-trust.pem -verify_return_error </dev/null \| grep -E "subject=\|issuer=\|Verify return code"` | Your certificate, and `Verify return code: 0 (ok)` |
| WSO2 is healthy | `curl -sf --cacert wso2-trust.pem https://<nlb-dns-name>/api/health-check/v1.0/health` | Success |
| Token issuer | `curl -s --cacert wso2-trust.pem https://<nlb-dns-name>/oauth2/token/.well-known/openid-configuration \| jq -r .issuer` | `https://<nlb-dns-name>/oauth2/token`, with no port |
| Your signing key is used | `curl -s --cacert wso2-trust.pem https://<nlb-dns-name>/oauth2/jwks \| jq -r '.keys[0].x5c[0]'`, compared with `grep -v CERTIFICATE signing.crt \| tr -d '\n'` | The two values are the same |
| Console sign-in | Open `https://<nlb-dns-name>/console` in a browser on a device that trusts your certificate, and sign in as the super admin | The Console opens with no certificate warning |
| WSO2 is connected to your server | On the database server: `SELECT datname, count(*) FROM pg_stat_activity WHERE usename = 'wso2' GROUP BY datname;` | Connections to all three databases |
| Connections use TLS | `SELECT a.datname, s.ssl FROM pg_stat_ssl s JOIN pg_stat_activity a USING (pid) WHERE a.usename = 'wso2';` | `ssl` is `t` for every row |
| Data is saved | Create an application in the Console, then run `psql -d wso2_identity -c "select count(*) from IDN_OAUTH_CONSUMER_APPS"` | The count goes up |
| Agents are saved | Create an agent (Console → Agents), then run `psql -d wso2_agent -c "select count(*) from UM_USER"` | The count goes up |
| Data survives a restart | `kubectl -n wso2 rollout restart deploy/wso2is`, then look for the same application and agent | Both are still there; the issuer and the signing key haven't changed |

# 10. Looking after it

- **Backups:** include the three WSO2 databases in your PostgreSQL backups. Keep the keystores and their password in Secrets Manager.
- **Database maintenance:** restarting or upgrading the PostgreSQL server interrupts WSO2.
- **Renewing the certificate:**
    1. Option A: re-import it into ACM under the same ARN (`aws acm import-certificate --certificate-arn …`).
    2. Option B: rebuild `tls.p12`, update the secret, and restart the pod (`kubectl -n wso2 rollout restart deploy/wso2is`).
    3. If the certificate is self-signed, also update the `wso2-trust` ConfigMap and every device that trusts the old certificate, before it expires.
- **Keep the NLB:** its DNS name is WSO2's address (§8.1).
- **Changing the signing key:** tokens signed with the old key stop working. Plan it for a quiet time.
- **Upgrading WSO2:** follow WSO2's migration guide for the new version, back up the databases first, and repeat the checks in §9.
- **More than one pod:** this needs WSO2 clustering. Stay with one pod for dev.
- **Logs:** don't turn on OAuth debug logging on a shared environment. Debug logs can contain tokens.
- **Super admin:** use it only for administration, and keep its password in Secrets Manager.

# 11. Troubleshooting

| Problem | Cause and fix |
|---|---|
| The pod stops at once with `cp: cannot open '/keys/…': Permission denied` | WSO2 can't read the key files. Set `fsGroup: 802` and `defaultMode: 0440` (§7) |
| `ClassNotFoundException: org.postgresql.Driver` | The PostgreSQL driver is missing. Use the custom image (§5) |
| `Connection refused` or a time-out to the database | A security group, firewall or route is blocking the pod. Test from a pod in the same namespace: `kubectl -n wso2 run pg-test --rm -it --image=postgres:16 -- psql "host=<db-host> port=<db-port> user=wso2 dbname=wso2_shared sslmode=require"` |
| `PKIX path building failed` or `certificate verify failed` for the database | `db-ca.pem` is the wrong authority, or `<db-host>` isn't a name in the database's certificate. Use the name the certificate was issued for |
| `no pg_hba.conf entry for host …` | The server doesn't allow the `wso2` user from the pods' addresses. Ask the database administrator to add the EKS network for the three databases, with TLS |

| Problem | Cause and fix |
| --- | --- |
| `remaining connection slots are reserved` or `too many clients` | The server is out of connections. Add pool limits (§6), or ask for a higher `max_connections` |
| Data disappears after a restart | The ConfigMap isn't mounted at `wso2-config-volume/…`, so WSO2 is still using H2. If only agents disappear, the `[datasource.AgentIdentity]` section is missing |
| `relation "um_user" already exists` while loading the agent tables | The agent script was run in the shared database. Agents need their own database (§3.1) |
| The issuer or redirects show `:9443` | `proxyPort = 443` is missing (§6) |
| Clients report a certificate name mismatch | They're using an address your wildcard doesn't cover, such as an ALB name or the Kubernetes Service name. Use `<nlb-dns-name>` |
| The NLB's targets stay unhealthy | Check that the health check uses HTTPS and the path in §8.1, and that the pod is ready |
| Pods inside the cluster time out when calling WSO2 through the NLB | A client can't reach itself through an NLB that keeps the client's IP address. Add `service.beta.kubernetes.io/aws-load-balancer-target-group-attributes: preserve_client_ip.enabled=false` to the NLB Service |
| Everything fails after a redeploy, with issuer or redirect errors | The NLB was recreated with a new name, so WSO2's address changed. Put the new name in `deployment.toml`, restart WSO2, and update anything set up with the old address. Keep the NLB to avoid this (§8.1) |
| Clients fail with "self-signed certificate", "unable to get local issuer certificate" or `PKIX path building failed` | The client doesn't trust `wso2-trust.pem` (§2.1). If a CA issued your certificate, the chain may also be missing: include `chain.pem` in `tls.p12` (Option B), or import ACM with `--certificate-chain` (Option A) |
| Browsers show a certificate warning | The device doesn't trust your certificate. Install `wso2-trust.pem` through your device management. Don't tell users to click through the warning |
| WSO2 can't connect to an identity provider or other internal service you added | WSO2 doesn't trust that service's certificate. Import it, or its CA, in `start.sh` under its own alias (§5) |
| The super admin's new password doesn't work | The super admin is created on first start only. Changing the secret later doesn't change the password; change it in the Console |

The `pg-test` command in the table uses `sslmode=require` only to test the network path quickly. WSO2 itself always uses `verify-full`.